

KhaanQuest Internal iOS Inventory System — Sprint Plan

Project Technology Stack

- iOS Frontend: Swift, SwiftUI, Xcode
 - Local Storage: SwiftData
 - Backend: FastAPI, Python
 - Database: PostgreSQL, Alembic
 - Cloud Services: AWS RDS, S3, ECR, App Runner or ECS
 - Test Distribution: TestFlight
 - Production Distribution: Apple Business Manager Private Custom App
 - Development Approach: API contract first, frontend and backend developed in parallel
-

Sprint 0: Project Foundation and Infrastructure

Duration: 1 week

Sprint Goal

Set up the development, testing, and deployment foundations so the iOS app, backend, and database can communicate successfully.

iOS Frontend

- Create the Xcode SwiftUI project
- Configure Development, Staging, and Production environments
- Create the initial project folder structure
- Build the base design system
- Define colors, typography, buttons, cards, and status components
- Create the bottom tab navigation
- Create shared loading, empty, and error states
- Create the mock data framework

Backend

- Create the FastAPI project
- Create the API v1 routing structure
- Configure environment variables and settings
- Define a standard API response format
- Add global exception handling

- Configure application logging
- Add a health check endpoint at `/health`

Database and Cloud

- Create the local PostgreSQL development database
- Configure SQLAlchemy
- Configure Alembic
- Create the Development database
- Create the Staging database
- Configure the AWS account and basic permissions
- Create the initial S3 product image storage plan

DevOps

- Create the Git repository
- Create `main` and `develop` branches
- Configure Pull Request rules
- Configure automated backend tests
- Configure automated iOS build checks
- Write the README and local setup instructions

Acceptance Criteria

- The iOS app launches and displays the base navigation
- FastAPI runs locally
- `/health` returns a successful response
- FastAPI connects to PostgreSQL
- Alembic migrations run successfully
- The iOS app can call the test API

Sprint 1: Authentication, Roles, and Profile

Duration: 2 weeks

Sprint Goal

Complete user authentication, role-based access control, and the basic Profile section.

iOS Frontend

- Login screen
- Persistent login state
- Secure token storage using Keychain

- Automatic access token refresh
- Logout
- Profile screen
- Display user name, role, and assigned locations
- Display current working location
- Display sync status
- Display app version

Backend

- Login endpoint
- Access token and refresh token handling
- Password hashing
- Current user endpoint
- Logout and token invalidation
- Role and permission middleware
- User active and disabled status

Database

- users
- roles
- user_roles
- user_locations
- refresh_tokens
- devices
- audit_logs

Testing

- Successful login
- Invalid password
- Disabled user cannot log in
- Expired token refresh
- Permission checks for different roles
- Token becomes invalid after logout

Acceptance Criteria

- Users can log in and log out
 - Login remains active after reopening the app
 - Users can only access authorised locations
 - The Profile page displays the correct role and location
 - Login activity is written to the audit log
-

Sprint 2: SKU, Product Images, and Locations

Duration: 2 weeks

Sprint Goal

Complete product master data and inventory location management.

iOS Frontend

- SKU list
- SKU fuzzy search
- SKU details
- Create SKU
- Edit SKU
- Product image placeholder
- Add, replace, and delete images later
- Location list
- Location details
- Display locations based on user permissions

Backend

- SKU CRUD endpoints
- SKU search endpoint
- SKU uniqueness validation
- Product image upload signing endpoint
- Warehouse and event location endpoints
- Product activation and deactivation
- Reserved endpoint for bulk SKU import

Database

- products
- product_variants
- product_images
- locations
- location_types

Testing

- Duplicate SKU is rejected
- SKU works without an image
- Images can be added later without affecting inventory
- Disabled SKU cannot be used in new operations

- Users can only access authorised locations

Acceptance Criteria

- Users can create, search, and edit SKUs
 - Product images are optional
 - Images can be added after SKU creation
 - China, Australia, United States, and event locations are clearly distinguished
-

Sprint 3: Core Inventory and Inventory Movements

Duration: 2 weeks

Sprint Goal

Build the core inventory balance and inventory movement logic.

iOS Frontend

- Global inventory dashboard
- Inventory list by location
- SKU inventory details by location
- Stock receipt screen
- Inventory adjustment screen
- Damage, giveaway, and return operations
- Inventory movement list
- Inventory movement details
- Low-stock and out-of-stock status display

Backend

- Inventory balance query
- Stock receipt
- Inventory adjustment
- Damaged stock
- Giveaway stock
- Return to stock
- Inventory movement creation
- Database transactions
- Concurrent inventory locking
- Idempotent request handling
- Audit logging

Database

- `inventory_levels`
- `inventory_movements`
- `inventory_adjustments`
- `idempotency_keys`

Core Rules

- Available = On Hand – Committed – Unavailable
- Sellable = Available – Safety Stock
- Every inventory change must create an inventory movement
- Inventory movements cannot be deleted
- Errors must be corrected using reverse movements
- One SKU can only have one inventory balance record per location

Testing

- Successful stock receipt
- Insufficient inventory
- Duplicate request
- Concurrent inventory deduction
- Transaction rollback on failure
- Inventory movements match inventory balances

Acceptance Criteria

- Every inventory change has a complete movement record
- Negative inventory is blocked unless explicitly allowed
- Duplicate requests do not deduct inventory twice
- Inventory balances can be reconciled against inventory movements

Sprint 4: Multi-SKU Transfers

Duration: 2 weeks

Sprint Goal

Complete multi-SKU stock transfers between China, Australia, the United States, and event locations.

iOS Frontend

- Create transfer order

- Select source and destination locations
- Search and add multiple SKUs
- Edit transfer quantity for each SKU
- Remove an SKU
- Display total SKU count and total units
- Transfer details
- Approve transfer
- Confirm shipment
- Enter tracking number
- Enter received quantity for each SKU
- Partial receipt
- Full receipt
- Display shortages, damages, and discrepancies

Backend

- Create multi-SKU transfer
- Approve transfer
- Ship transfer
- Partially receive transfer
- Fully receive transfer
- Cancel unshipped transfer
- Validate transfer state transitions
- Deduct inventory from the source location
- Increase incoming inventory at the destination
- Increase destination inventory after receipt
- Record shortages and damages

Database

- transfers
- transfer_items
- transfer_receipts
- transfer_receipt_items

Transfer Statuses

- DRAFT
- APPROVED
- IN_TRANSIT
- PARTIALLY_RECEIVED
- RECEIVED
- CANCELLED

Testing

- One transfer contains multiple SKUs

- Duplicate SKU cannot be added twice
- Quantity cannot exceed available inventory
- Some SKUs can be received before others
- The same SKU can be received in multiple batches
- Shipped transfers cannot be directly cancelled
- Transfer discrepancies are recorded correctly

Acceptance Criteria

- A transfer can contain multiple SKUs
 - All items in one transfer must share the same source and destination locations
 - Each SKU can be received independently
 - The transfer becomes RECEIVED only when all items are fully received
-

Sprint 5: Event Inventory and Online Sales

Duration: 2 weeks

Sprint Goal

Complete Rodeo, exhibition, and market event inventory and online sales workflows.

iOS Frontend

- Event list
- Create event
- Event details
- Temporary event location
- Allocate stock from the Australia warehouse to the event
- Event inventory list
- Manual SKU entry and search
- Recently used SKUs
- Multi-SKU sales order
- Edit quantity and price
- Payment method
- Giveaway
- Return
- Event sales summary
- Return remaining event inventory to the warehouse

Backend

- Create and close event
- Create event inventory location

- Allocate event stock
- Create sales order
- Multi-SKU sales
- Payment recording
- Returns
- Giveaways
- Event reconciliation
- Return inventory to the warehouse

Database

- events
- event_assignments
- sales
- sale_items
- payments
- returns
- return_items

Testing

- Multi-SKU sale
- Duplicate SKU lines are merged
- Different payment methods
- Sale deducts event inventory
- Return restores inventory
- Closed event cannot accept new sales
- Remaining event stock returns correctly

Acceptance Criteria

- A full event workflow can be completed from allocation to sales, returns, and stock return
- Event inventory matches sales activity
- Every sale records the staff member and payment method

Sprint 6: Offline Sales and Synchronisation

Duration: 2 weeks

Sprint Goal

Ensure the app continues to work at Rodeo events when there is no internet connection.

iOS Frontend

- SwiftData local database
- Download event SKU and inventory snapshot
- Offline sales
- Offline giveaways
- Offline returns
- Offline stock count preparation
- Pending sync queue
- Sync status display
- Manual sync
- Automatic retry
- Sync failure details
- Automatic sync when the network returns
- Unique device identifier

Backend

- Batch sync endpoint
- Deduplication using `client_operation_id`
- Device identification
- Per-operation sync results
- Duplicate operation detection
- Inventory conflict detection
- Safe automatic retry
- Conflict logging
- Administrator conflict resolution endpoint

Database

- `sync_operations`
- `sync_conflicts`
- `devices`
- `idempotency_keys`

Sync Statuses

- PENDING
- SYNCING
- SYNCED
- FAILED
- CONFLICT

Testing

- Create 100 offline sales

- Data survives app restart
- All records upload after network recovery
- Duplicate upload does not deduct inventory twice
- Two devices sell the final available unit
- Some operations succeed while others fail
- Network disconnects during synchronisation

Acceptance Criteria

- Offline sales are not lost
 - Repeated sync does not duplicate inventory deductions
 - Conflicting operations are identified and recorded
 - Users can clearly see the sync status of every operation
-

Sprint 7: Stock Counting

Duration: 2 weeks

Sprint Goal

Complete warehouse and event stock counting and discrepancy handling.

iOS Frontend

- Create stock count
- Select inventory location
- Search or manually enter SKU
- Display expected quantity
- Enter counted quantity
- Automatically calculate variance
- Save stock count draft
- Display progress
- Submit stock count
- Administrator variance approval
- Offline stock count
- Stock count history

Backend

- Create stock count task
- Save stock count items
- Submit stock count
- Calculate variance
- Approve or reject variance

- Generate inventory adjustment movements
- Prevent overlapping stock counts for the same location

Database

- `stock_counts`
- `stock_count_items`
- `stock_count_approvals`

Testing

- Expected quantity matches counted quantity
- Positive variance
- Negative variance
- Draft restoration
- Offline stock count sync
- Administrator rejects variance
- Inventory updates correctly after approval

Acceptance Criteria

- Stock count variance must be approved before changing official inventory
- Every approved adjustment creates an inventory movement
- Stock count progress is not lost

Sprint 8: Inventory Analytics, Alerts, and Notifications

Duration: 2 weeks

Sprint Goal

Complete the inventory dashboard, fastest-consuming SKU analysis, and notification system.

iOS Frontend

- Top 10 fastest-consuming SKUs
- 7-day, 30-day, and 90-day filters
- Custom date range
- Net sales consumption
- Average daily consumption
- Estimated days of stock remaining
- Out-of-stock list

- Low-stock list
- Suggested replenishment quantity
- Notification centre
- Unread notification count
- Notification preferences

Backend

- Fastest-consuming SKU query
- Average daily consumption calculation
- Estimated days of stock calculation
- Reorder point calculation
- Suggested reorder calculation
- Out-of-stock and low-stock alerts
- Notification generation
- Mark notification as read
- Scheduled analytics jobs

Database

- `inventory_alerts`
- `notifications`
- `notification_preferences`

Calculation Rules

- Net Consumption = Sales Quantity – Sellable Return Quantity
- Transfers are excluded from global sales consumption
- Estimated Days Remaining = Current Sellable Inventory ÷ Average Daily Net Consumption
- Suggested Reorder = Target Stock – Sellable – Incoming

Testing

- Returns correctly reduce consumption
- Transfers do not count as sales consumption
- No division-by-zero error when there are no sales
- Low-stock notifications are not repeatedly duplicated
- Alerts are calculated separately for each location

Acceptance Criteria

- Inventory alerts are visible on the dashboard
- Users can view the fastest-consuming SKUs
- Data can be filtered by global inventory, location, or event
- Notifications can be marked as read

Sprint 9: Administration and System Management

Duration: 2 weeks

Sprint Goal

Complete internal administration features and security controls.

iOS Frontend

- User management
- Create and disable users
- Role management
- User location permissions
- Location management
- Event staff assignment
- Alert rule configuration
- Data export
- Audit log viewer
- System settings
- Help and feedback

Backend

- User management endpoints
- Role and permission endpoints
- Location permission endpoints
- Data export
- Audit log search
- Additional confirmation for sensitive actions
- Sensitive operation access control
- Failed login attempt limits

Database

- Complete role and permission tables
- Administrator settings
- Export job records
- Security event records

Testing

- Standard users cannot access administrator features
- China warehouse staff cannot modify Australia inventory
- Administrator actions are fully logged

- Disabled users lose access immediately
- Data export permission checks

Acceptance Criteria

- Administrators can manage users and permissions
 - Sensitive operations are fully traceable
 - Standard users only access authorised functions
-

Sprint 10: Shopify Integration

Duration: 2 weeks

Sprint Goal

Connect Shopify orders and inventory while preventing duplicate stock deductions.

iOS Frontend

- Shopify connection status
- Sync success and failure messages
- Shopify order source label
- SKU mapping exception list
- Manual retry

Backend

- Shopify Webhook receiver
- Webhook signature verification
- Order creation
- Order cancellation
- Refund handling
- SKU mapping
- Location mapping
- Webhook idempotency
- Inventory sync
- Failed sync retry

Database

- `shopify_connections`
- `shopify_product_mappings`
- `shopify_location_mappings`
- `shopify_webhook_events`

- `integration_failures`

Testing

- Duplicate Webhook does not deduct stock twice
- Shopify orders map to the correct SKU
- Unknown SKU enters the exception queue
- Refund correctly restores inventory
- Failed synchronisation can be retried

Acceptance Criteria

- Shopify orders automatically enter the system
 - The same order never deducts stock twice
 - Integration failures can be tracked and retried
-

Sprint 11: System Testing and Pilot Operation

Duration: 2 weeks

Sprint Goal

Complete end-to-end testing and run the system during a real small event.

Test Scope

- Authentication and permissions
- SKU management
- Stock receipt
- Multi-SKU transfer
- Partial receipt
- Event stock allocation
- Online sales
- Offline sales
- Offline synchronisation
- Returns
- Stock counting
- Inventory analytics
- Shopify integration
- Database backup and restore

Real Business Simulation

- China warehouse creates a multi-SKU transfer

- China warehouse confirms shipment
- Australia warehouse partially receives the transfer
- Australia warehouse receives the remaining items
- Australia warehouse allocates stock to an event
- Rodeo staff complete offline sales
- The app synchronises after the network returns
- Event stock is counted after the event
- Remaining inventory returns to the Australia warehouse
- Inventory movements and sales totals are reconciled

Acceptance Criteria

- No blocking defects
 - Inventory balances match inventory movements
 - Offline data is not lost
 - No permission escalation vulnerabilities
 - A real event can complete the full workflow
 - The database can be successfully backed up and restored
-

Sprint 12: Internal Production Launch

Duration: 1–2 weeks

Sprint Goal

Complete the production cloud environment, private internal app distribution, and staff onboarding.

Cloud Deployment

- Create Production RDS PostgreSQL
- Configure automated database backups
- Deploy Production FastAPI
- Configure HTTPS and domain
- Configure S3 image storage
- Configure logging and monitoring
- Configure alerts
- Run production database migrations

iOS Release

- Create the production App Bundle ID
- Configure signing
- Upload to App Store Connect
- Complete final TestFlight validation

- Configure Apple Business Manager
- Release as a Private Custom App
- Invite internal users to install the app

Business Launch

- Import official SKU data
- Import official warehouse inventory
- Create official staff accounts
- Configure user permissions
- Train staff
- Publish the operations manual
- Assign technical support responsibility
- Establish the issue reporting process

Acceptance Criteria

- Internal users can install the app
- Users can log in to the production environment
- Official inventory is imported accurately
- Automated backups work correctly
- Monitoring and alerts are active
- The first official business transaction is completed

Definition of Done for Every Sprint

A task is considered complete only when:

- Requirements and acceptance criteria are clear
- The OpenAPI specification is updated
- Database migrations are committed
- Backend implementation is complete
- iOS implementation is complete
- Unit tests pass
- API tests pass
- iOS build passes
- Permission tests pass
- Frontend and backend integration is complete
- Staging validation is complete
- There are no critical defects
- Operational documentation is updated
- The Pull Request has been reviewed and merged

Estimated Overall Timeline

Phase	Sprint	Duration
Project foundation	Sprint 0	1 week
Core platform	Sprint 1–3	6 weeks
Transfers and events	Sprint 4–5	4 weeks
Offline and stock count	Sprint 6–7	4 weeks
Analytics and administration	Sprint 8–9	4 weeks
Shopify	Sprint 10	2 weeks
Testing and launch	Sprint 11–12	3–4 weeks

Estimated total duration: approximately 24–25 weeks.

This is a realistic schedule for one developer working part-time or close to full-time. The system can begin internal pilot testing after Sprint 9 if Shopify integration is postponed.